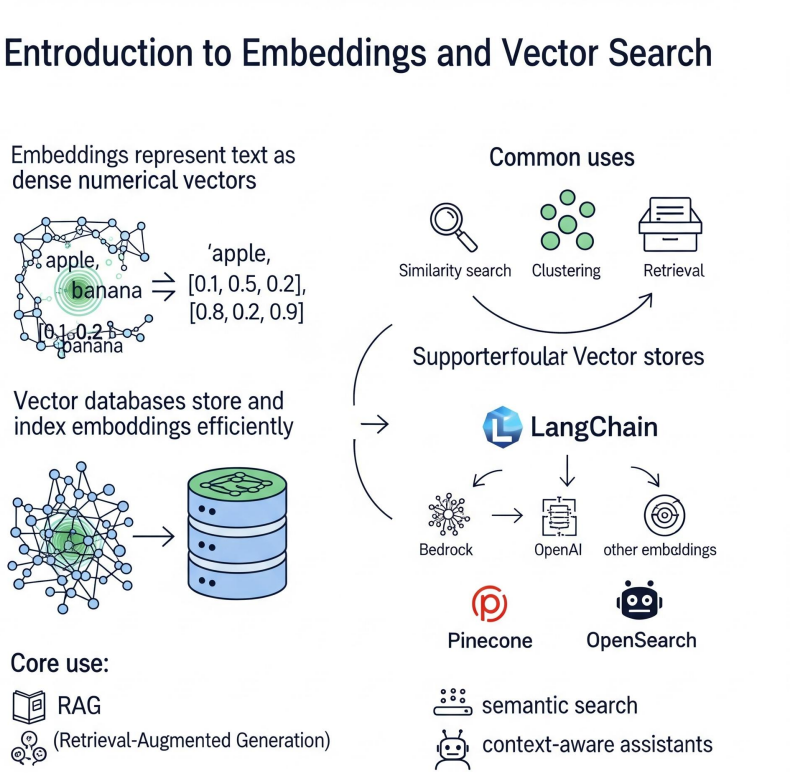
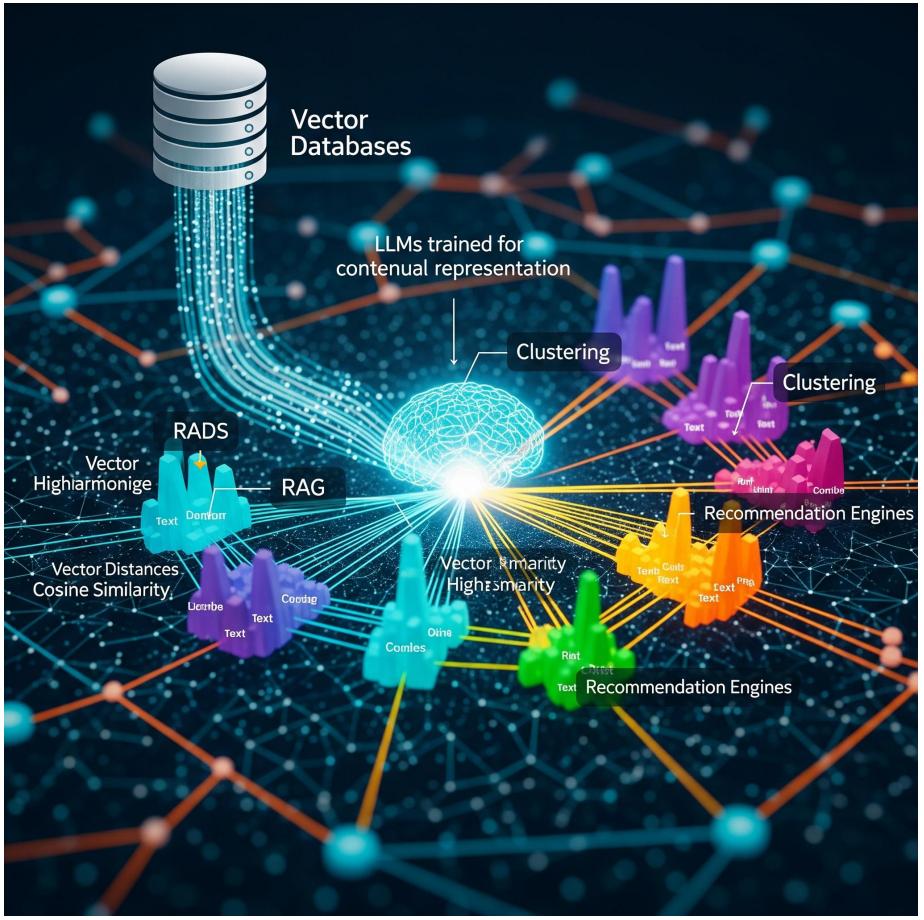
Introduction to Embeddings and Vector Search

- **Embeddings** represent text as dense numerical vectors.
- Used for similarity search, clustering, and retrieval.
- **Vector databases** store and index embeddings efficiently.
- **LangChain** supports Bedrock, OpenAI, and other embeddings.
- **Pinecone** and OpenSearch are popular vector stores.
- **Core use:** RAG, semantic search, and context-aware assistants.



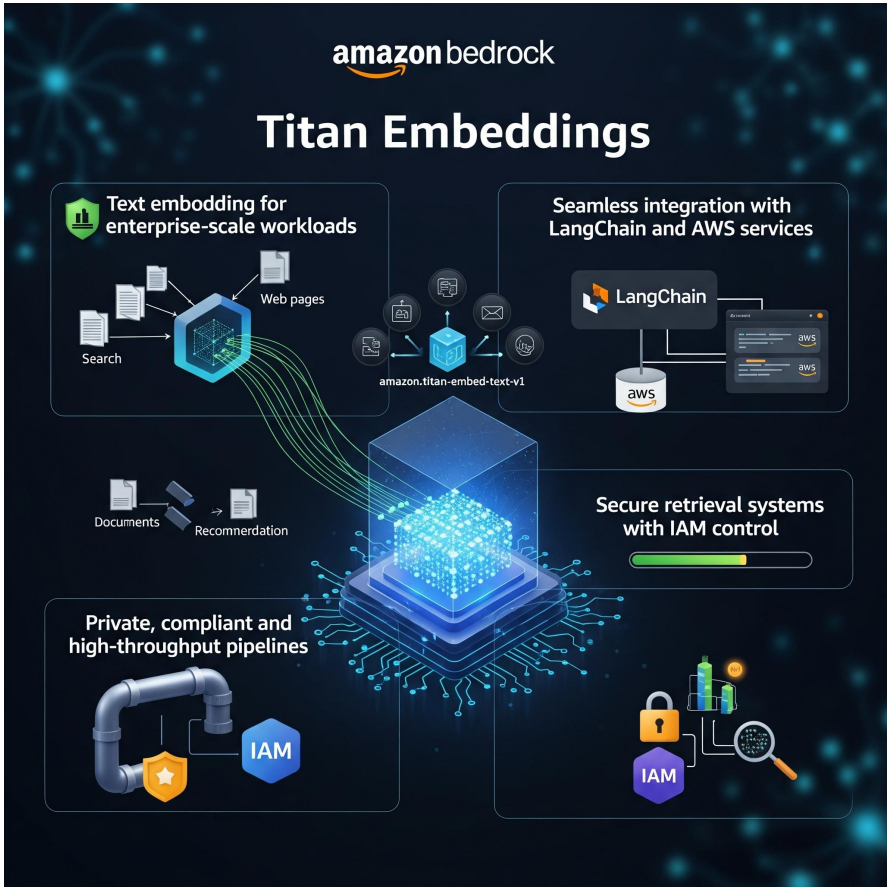
What Are Embeddings?

- High-dimensional representations of semantic meaning.
- Similar texts have closer vector distances (cosine similarity).
- Enable retrieval of related documents beyond keyword search.
- Computed by LLMs trained for contextual representation.
- Stored in specialized vector databases for fast retrieval.
- Used in RAG, clustering, and recommendation engines.



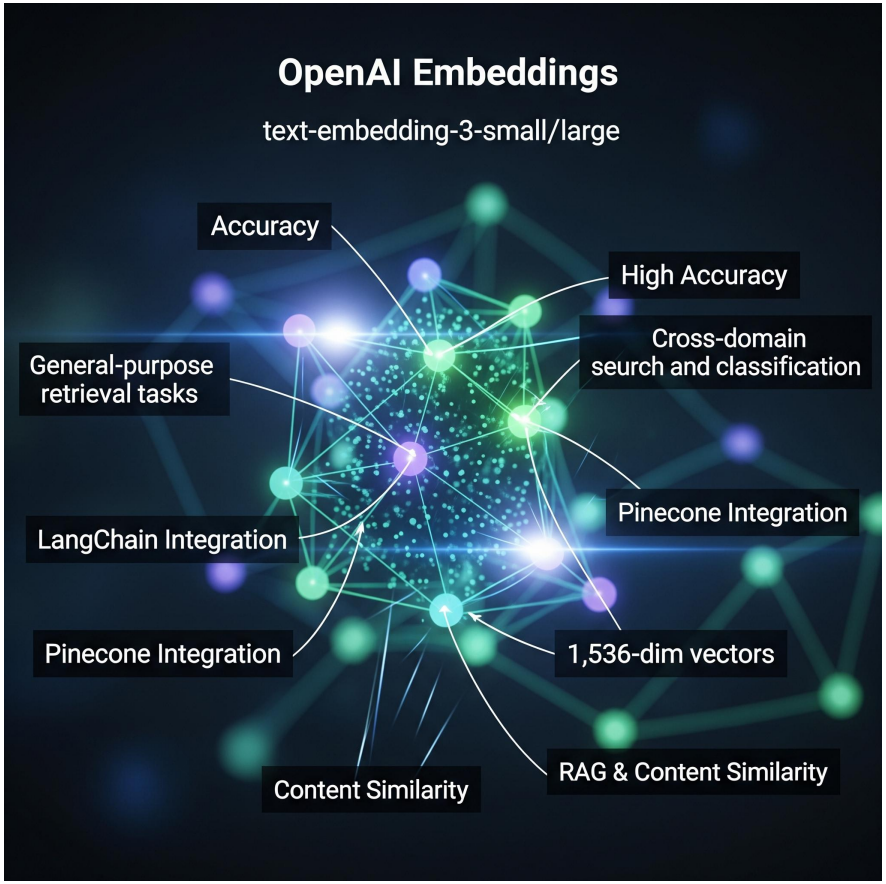
Amazon Bedrock Titan Embeddings

- Titan provides embedding models via Amazon Bedrock.
- Supports text embedding for enterprise-scale workloads.
- Integrates seamlessly with LangChain and AWS services.
- Example model: 'amazon.titan-embed-text-v1'.
- Ideal for private, compliant, and high-throughput pipelines.
- Lets you build secure retrieval systems with IAM control.



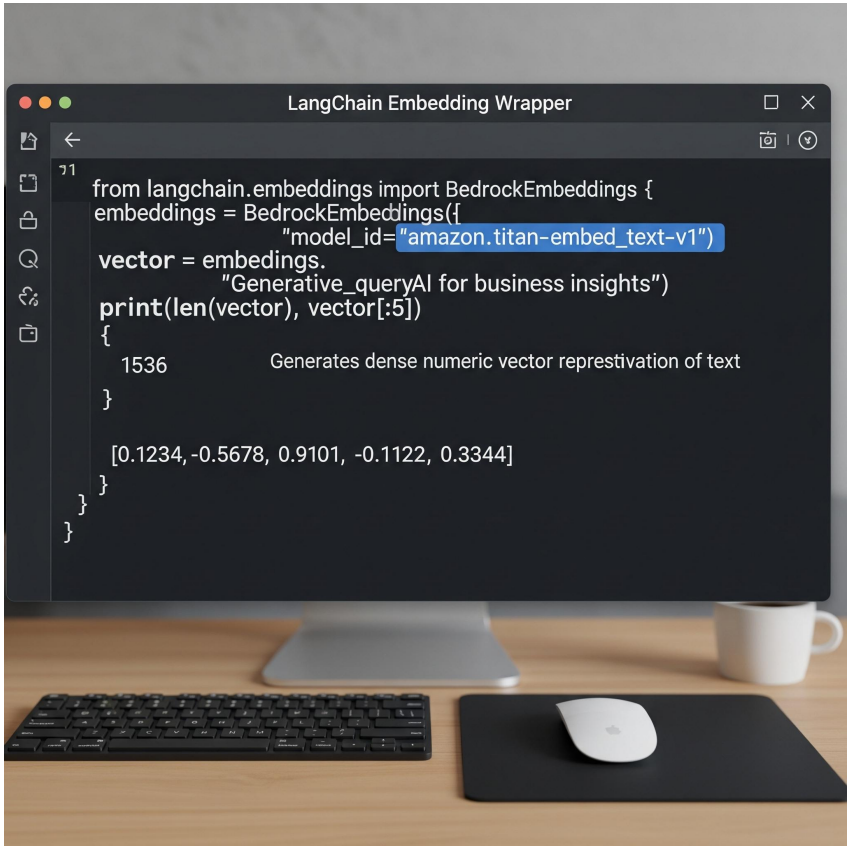
OpenAI Embeddings

- OpenAI embeddings via models like text-embedding-3-small/large.
- Provide high accuracy for general-purpose retrieval tasks.
- Integrate easily with LangChain and Pinecone.
- Useful for cross-domain search and classification tasks.
- Return 1,536-dim vectors representing input semantics.
- Well-suited for RAG and content similarity detection.



LangChain Embedding Wrapper Example

```
from langchain.embeddings import BedrockEmbeddings
embeddings = BedrockEmbeddings(model_id='amazon.titan-embed-text-v1')
vector = embeddings.embed_query('Generative AI for business insights')
print(len(vector), vector[:5])
```



- Generates dense numeric vector representation of text.

Embedding Generation Comparison

Embedding Generation Comparison

Features	Bedrock Titan	OpenAI	Cohere
Security	IAM	API Key	API Key
Context Length	8K	8K	API Key
Latency	Low	Medium	4K
Cost	Low	Medium	–
Integration	AWS billing	Usage-based	Global
Integration	AWS-native	Global	General

Vector Stores Overview

- Vector databases store and retrieve embeddings efficiently.
- Use distance metrics like cosine or Euclidean similarity.
- Popular choices: Pinecone, OpenSearch, FAISS, Chroma.
- Support metadata filtering and hybrid search.
- LangChain abstracts vector store interaction via retrievers.
- Foundation for Retrieval-Augmented Generation (RAG).

Pinecone Overview

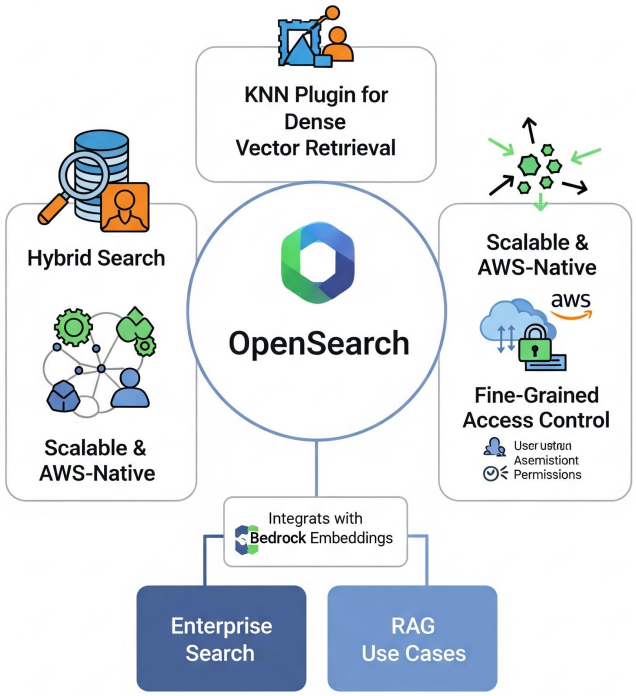
- Pinecone: managed vector database with high scalability.
- Provides APIs for vector upsert, query, and metadata filtering.
- Integrates natively with LangChain for retrieval tasks.
- Automatic sharding and replication for high availability.
- Supports billions of vectors with millisecond latency.
- Ideal for global-scale enterprise deployments.

Pinecone Integration Example

```
from langchain.vectorstores import Pinecone
from pinecone import Pinecone as PC
pc = PC(api_key='YOUR_API_KEY')
index = pc.Index('ai-knowledge-base')
vectorstore = Pinecone(index, embeddings.embed_query, 'text')
results = vectorstore.similarity_search('AI in retail')
print(results[0].page_content)
```

OpenSearch Overview

- OpenSearch = open-source alternative for vector indexing.
- Combines full-text + semantic vector search (hybrid search).
- Supports KNN plugin for dense vector retrieval.
- Scalable and AWS-native with fine-grained access control.
- Good fit for enterprise search and RAG use cases.
- Integrates well with Bedrock embeddings.

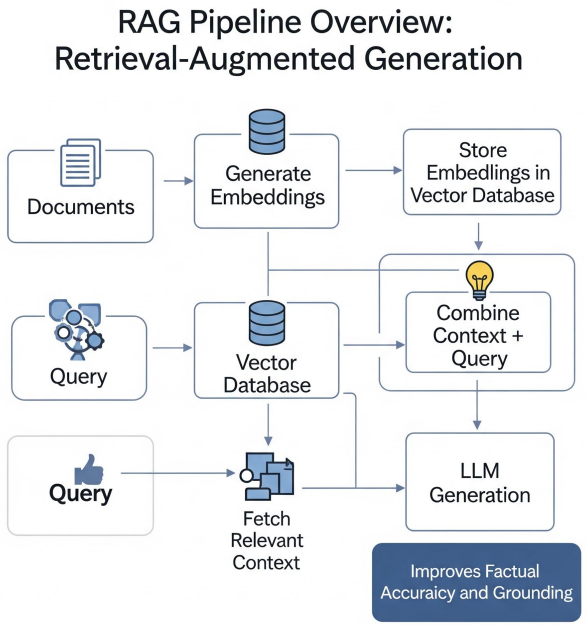


OpenSearch Integration Example

```
from langchain.vectorstores import OpenSearchVectorSearch
vectorstore = OpenSearchVectorSearch(
 index_name='llm-knowledge',
embedding_function=embeddings.embed_query,
 opensearch_url='https://opensearch.local:9200')
vectorstore.add_texts(['AI transforms enterprise workflows'])
results = vectorstore.similarity_search('enterprise AI')
print(results)
```

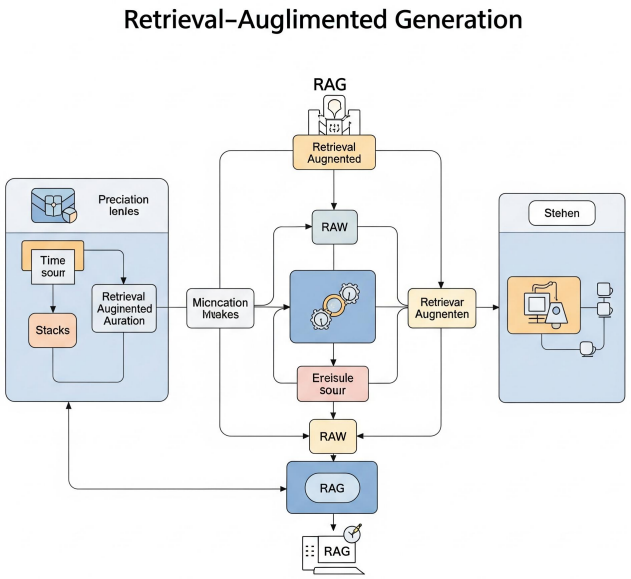
RAG Pipeline Overview

- RAG = Retrieval-Augmented Generation.
- 1. Generate embeddings from documents.
- 2. Store embeddings in vector database.
- 3. Query vector DB to fetch relevant context.
- 4. Combine context + query for LLM generation.
- Improves factual accuracy and grounding.



RAG Workflow Example

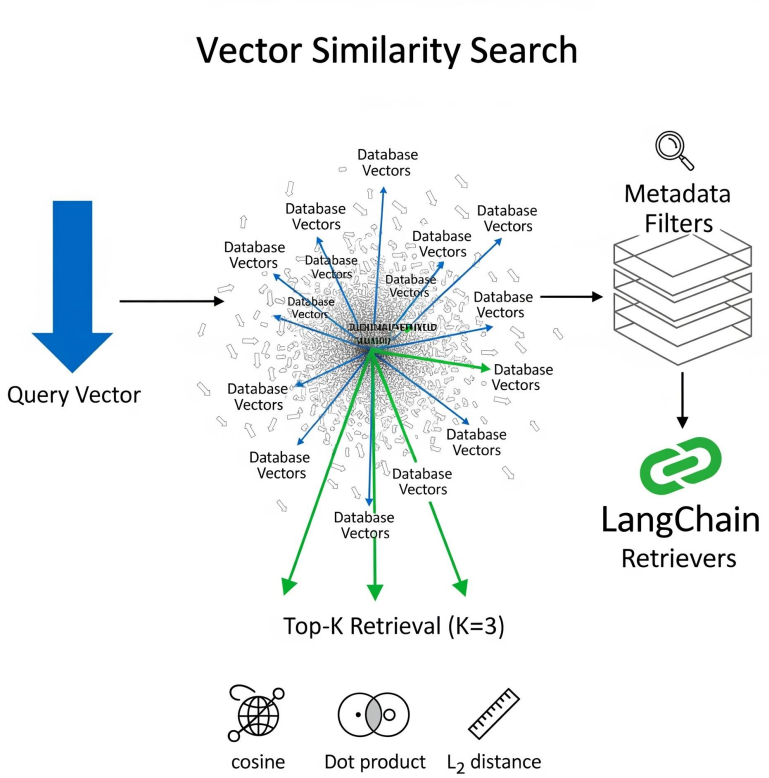
```
from langchain.chains import RetrievalQA
retriever = vectorstore.as_retriever(search_kwargs={'k': 3})
qa = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
response = qa.run('What are key uses of Bedrock Titan?')
print(response)
```



- Fetches context-aware answers grounded in stored knowledge.

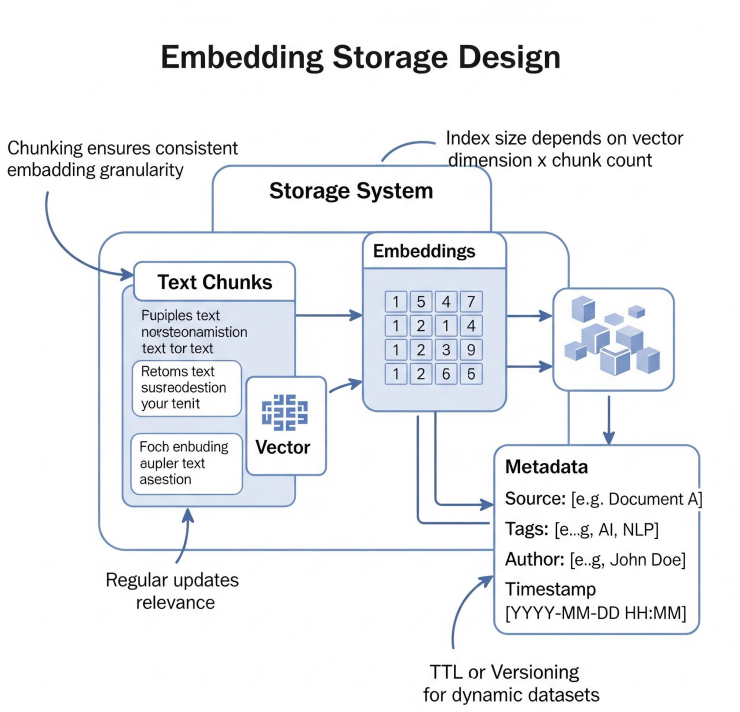
Vector Similarity Search

- Similarity search uses distance metrics between embeddings.
- Common metrics: cosine, dot product, L2 distance.
- Top-K retrieval: selects K nearest vectors to query.
- Metadata filters enable structured search.
- LangChain retrievers manage search automatically.
- Improves speed and contextual precision.



Embedding Storage Design

- Store text chunks + embeddings + metadata.
- Metadata = source, tags, author, timestamp.
- Chunking ensures consistent embedding granularity.
- Index size depends on vector dimension × chunk count.
- Regular updates maintain relevance of stored data.
- Use TTL or versioning for dynamic datasets.

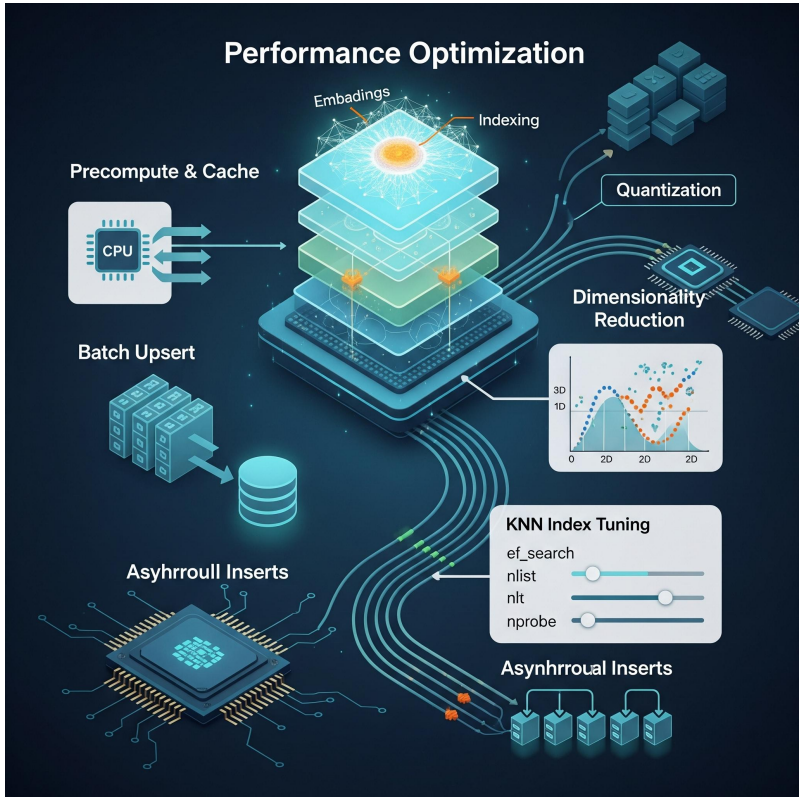


Vector DB Comparison Table

Feature	Pinecone	OpenSearch	FAISS	Best For
Type	Cloud-native	OpenSearch	Library	Managed high-scale
	Distributed Search Engine	–	–	Full-text & vector
Scalability	APISS	APIs, SDKs	–	–
Integration	Horizontal	Extensive Ecosystem	–	Application, dependent
Security	Amited	Role-based access, encryption	–	–
Security	Limited	Entregrise-grade	–	–
	(In-memory)	Python, C++	–	–

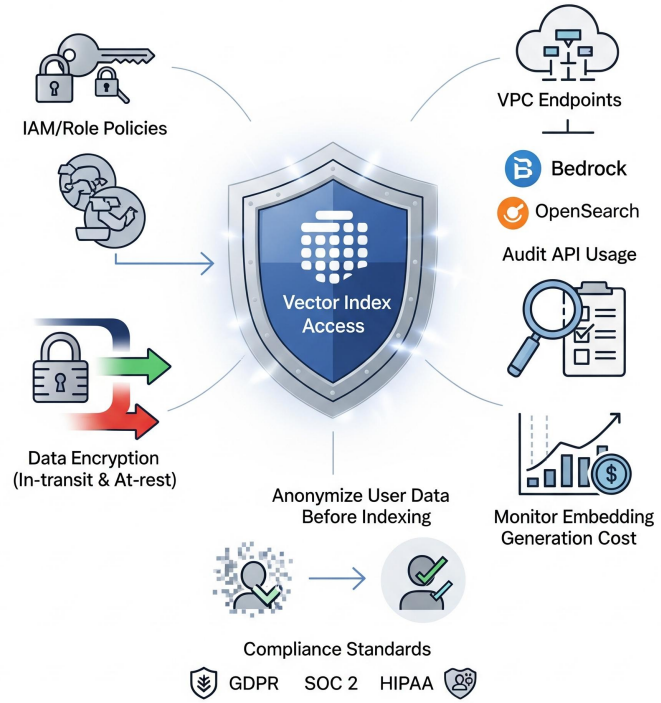
Performance Optimization

- Precompute and cache embeddings for frequent queries.
- Use batch upsert for efficient indexing.
- Apply dimensionality reduction for memory savings.
- Tune KNN index parameters (ef_search, nlist, nprobe).
- Enable asynchronous inserts for large-scale ingestion.
- Compress old data with quantization if needed.



Security & Governance

- Restrict vector index access with IAM or role policies.
- Encrypt data in transit and at rest.
- Use VPC endpoints for private Bedrock and OpenSearch.
- Audit API usage and monitor embedding generation cost.
- Anonymize user data before indexing.
- Follow compliance (GDPR, SOC 2, HIPAA) standards.



Best Practices

- Use Bedrock Titan for enterprise embeddings.
- Pinecone for fast, global vector search.
- OpenSearch for hybrid semantic + keyword retrieval.
- Regularly refresh embeddings for dynamic data.
- Leverage LangChain retrievers for modular pipelines.
- Monitor recall, precision, and latency metrics.

Summary

- Embeddings form the foundation for semantic retrieval.
- Vector databases power scalable, contextual RAG pipelines.
- LangChain unifies embedding generation and retrieval logic.
- Bedrock Titan, Pinecone, and OpenSearch complement each other.
- Apply hybrid search and metadata filters for better precision.
- Next: integrate embeddings into multi-agent AI workflows.